Software Defined Networks

Assignment 3

O-RAN Near-RT RIC Architecture & Software Components

and xApp Radio Measurement Subscriptions via E2SM-KPM

Group No. 19

Name	Roll No.
Kamal Nayan Kumar	23BDS026
Altaf Raja	23BCS011
Ankit Kumar Keshari	23BCS015
Soumya Nimje	23BEC054

Instructor

Dr. Jagadeesha R B

ಭಾರತೀಯ ಮಾಹಿತಿ ತಂತ್ರಜ್ಞಾನ ಸಂಸ್ಥೆ ಧಾರವಾಡ
भारतीय सूचना प्रौद्योगिकी संस्थान धारवाड़
Indian Institute of Information Technology Dharwad
[Institute of National Importance by An Act of Parliament]

Contents

1 Executive Summary **2**

2 Problem Statement **2**

3 O-RAN Architecture and Component Foundations **3**
3.1 O-RAN Disaggregation Overview . . . 3
3.2 Near-RT RIC Microservice Topology . . . 3
3.3 Deep Dive: Software Component Responsibilities . . . 4
3.3.1 E2 Manager (`e2mgr`) . . . 4
3.3.2 Subscription Manager (`submgr`) . . . 5
3.3.3 xApp Framework (`ric-plt-xapp-frame-py`) . . . 5

4 Objective 1: Installation and Software Component Inspection **5**
4.1 Environment Setup & Container Virtualization . . . 5
4.2 Multi-Container Platform Launch . . . 6
4.3 Network Topology & IP Configuration . . . 6
4.4 Component Log Analysis & Protocol Handshakes . . . 8

5 Objective 2: xApp Telemetry & Radio Measurement Subscriptions **8**
5.1 E2SM-KPM Service Model Overview . . . 8
5.2 Scenario 1: Uplink and Downlink Throughput Collection . . . 11
5.3 Scenario 2: Modification of Monitored Metrics . . . 11
5.4 Scenario 3: Advanced UE-Level Reporting (Report Style 5) . . . 13
5.5 Comparative Telemetry Summary . . . 13

6 Technical Discovery: Upstream APER Decoding Defect & Resolution **13**
6.1 Root Cause Analysis . . . 14
6.2 Remediation & Verification . . . 14

7 Conclusion **14**

1 Executive Summary

Open Radio Access Network (O-RAN) disaggregates the traditional, monolithic cellular base station into open, interoperable, and software-driven sub-components. In this paradigm, intelligence is introduced into the radio network through Software Defined Networking principles via Radio Intelligent Controllers (RICs). The Near-Real-Time RIC operates control loops between 10 ms and 1 second, hosting third-party microservices called **xApps** that execute radio resource management, mobility optimization, and real-time telemetry extraction.

This assignment investigates two objectives:

1. **Objective 1: Installation and Software Component Identification.** Deploy a minimal O-RAN Software Community (O-RAN SC) Near-RT RIC platform (`i-release`) in Docker. Inspect each microservice, verify inter-service communication over the RIC Message Router (RMR) and REST interfaces, and analyze the architectural responsibilities of the E2 Manager (`e2mgr`), Subscription Manager (`submgr`), and xApp Framework (`ric-plt-xapp-frame-py`).

2. **Objective 2: xApp Radio Measurement Subscription.** Analyze the E2 interface protocol stack (E2AP) and the E2 Service Model for Key Performance Measurements (E2SM-KPM). Deploy monitoring xApps inside the xApp runner container, subscribe to uplink and downlink user throughput metrics (`DRB.UEThpDl`, `DRB.UEThpUl`), modify the requested metrics to monitor RLC transmission volumes and reliability parameters, and evaluate multi-style telemetry reporting (Node-level vs. UE-level).

During implementation, an upstream software defect was identified in the official O-RAN SC Python framework (`ricxappframe`): the indication decoder incorrectly invoked Python's memory management inspection (`sys.getsizeof`) instead of buffer length (`len`), inducing intermittent ASN.1 Aligned Packed Encoding Rules (APER) decoding failures. We isolated the root cause, applied a patch, and verified full test suite compliance.

2 Problem Statement

Traditional cellular networks bundle radio hardware, baseband processing, and network control logic into proprietary base stations (eNodeB in 4G, gNodeB in 5G). This vendor lock-in prevents fine-grained network optimization and limits rapid service deployment. The O-RAN Alliance addresses this limitation by standardizing open interfaces and disaggregating the RAN into the Centralized Unit (O-CU), Distributed Unit (O-DU), and Radio Unit (O-RU), supervised by intelligent controllers.

The assignment tasks are specified as follows:

- **Task 1.1:** Verify the host container environment (Docker / container runtime) and launch the simplified O-RAN SC Near-RT RIC multi-container platform.

- **Task 1.2:** Inspect all running containers, examine network namespaces and port bindings, analyze startup logs across services, and explain the roles of the E2 Manager, Subscription Manager, and xApp Framework.

- **Task 1.3:** Examine the internal routing mechanism (RIC Message Router static routing table) and the Shared Data Layer (SDL) database backed by Redis.

- **Task 2.1:** Run the provided Python monitoring xApp, initiate an E2 subscription handshake targeting an active E2 node, and capture real-time uplink and downlink throughput measurements.

- **Task 2.2:** Reconfigure and modify the monitored metrics to observe layer-2 Radio Link Control (RLC) data volume, packet drop rates, and packet delivery success rates.
- **Task 2.3:** Explore advanced E2SM-KPM report structures, comparing E2-Node-level reporting (Report Style 1) against individual UE-level reporting (Report Style 5).

3 O-RAN Architecture and Component Foundations

3.1 O-RAN Disaggregation Overview

The O-RAN reference architecture establishes a hierarchical control structure divided into three distinct operational domains:

- **Non-Real-Time RIC (Non-RT RIC):** Located in the Service Management and Orchestration (SMO) framework. It operates on control loops greater than 1 second (> 1 s), hosting **rApps** that provide policy guidance, machine learning model training, and long-term network analytics over the **A1 interface**.
- **Near-Real-Time RIC (Near-RT RIC):** Deployed at the network edge close to the radio nodes. It operates on control loops between 10 ms and 1 second ($10\,\text{ms} \leq T \leq 1\,\text{s}$), hosting specialized micro-applications called **xApps**. The Near-RT RIC interfaces with baseband nodes over the standardized **E2 interface**.
- **Disaggregated E2 Nodes (O-CU and O-DU):** The O-CU handles higher-layer protocols (RRC, SDAP, PDCP), divided into O-CU-CP (Control Plane) and O-CU-UP (User Plane). The O-DU executes time-critical layers (RLC, MAC, High-PHY), while the O-RU handles Low-PHY and RF frontends over the Open Fronthaul (7-2x split) interface.

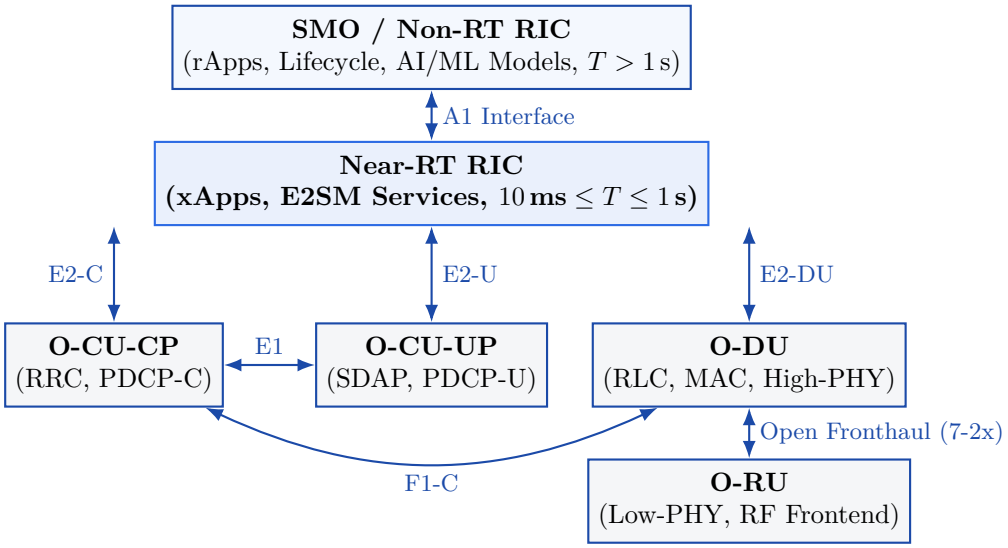


Figure 1: O-RAN reference architecture and functional disaggregation.

3.2 Near-RT RIC Microservice Topology

The O-RAN SC Near-RT RIC is structured as a collection of specialized microservices that communicate with each other using the **RIC Message Router (RMR)** for low-latency messaging, HTTP/REST for management and subscription workflows, and Redis for state persistence:

1. **E2 Termination (`e2term`):** Manages physical and transport connectivity with external E2 agents. It terminates SCTP connections on port `36421`, decodes incoming E2AP messages, and translates them into internal RMR messages.

2. **E2 Manager (`e2mgr`):** Maintains logical RAN connections and network topology. It processes node setup requests, tracks operational states in the Radio Network Information Base (RNIB), and directs routing table associations.

3. **Subscription Manager (`submgr`):** Coordinates E2 event subscriptions initiated by xApps. It maps xApp subscription identifiers to global E2 Event Instance IDs, maintains duplicate-free subscriptions to E2 nodes, and validates subscription states against the RNIB.

4. **Routing Manager Simulator (`rtmgr\_sim`):** Manages and distributes static routing tables (`routes.rtg`) across all microservices, ensuring that RMR messages route to the appropriate IP endpoints based on message type.

5. **Database as a Service (`dbaas`):** A Redis instance extended with SDL modules that stores state, including the Radio Network Information Base (RNIB) and subscription records.

6. **Application Manager (`appmgr`):** Provides lifecycle management (deployment, configuration, termination) for containerized xApps.

7. **Python xApp Runner (`python\_xapp\_runner`):** A dedicated execution container providing Python 3 bindings, ASN.1 compilers, RMR client libraries, and the official O-RAN SC Python framework.

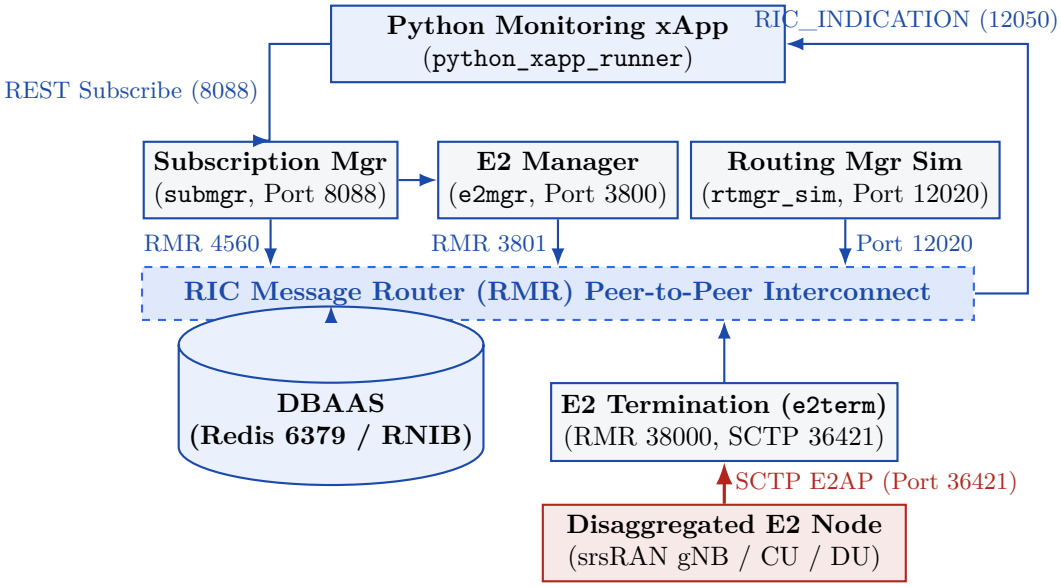


Figure 2: Internal microservice topology and message exchange channels of Near-RT RIC.

3.3 Deep Dive: Software Component Responsibilities

3.3.1 E2 Manager (`e2mgr`)

The E2 Manager is the central controller for E2 node inventory and connectivity. It operates on RMR port `3801` and provides a REST API on port `3800`. Its responsibilities comprise:

- **E2 Setup Coordination:** When a gNodeB boots, it sends an `E2SetupRequest` over SCTP to `e2term`. The termination point forwards this as RMR message `12001` (`RIC_E2_SETUP_REQ`) to `e2mgr`.

- **RNIB State Management:** `e2mgr` unpacks the RAN node identity (Global gNB ID, PLMN ID, supported RAN functions) and writes a structured entity record into Redis.

- **Routing Association:** It notifies the Routing Manager (`POST /ric/v1/handles/associate-ran-to-e2t`) to bind the logical RAN name (e.g., `gnb_001_001_0000019b`) to the associated E2 Termination address (`10.0.2.10:38000`).
- **Setup Completion:** It returns RMR message `12002` (`RIC_E2_SETUP_RESP`) through `e2term` to the base station.

3.3.2 Subscription Manager (`submgr`)

The Subscription Manager manages telemetry and event subscriptions requested by xApps. It operates a REST server on port `8088` and an RMR endpoint on port `4560`. Key responsibilities include:

- **API Ingestion & Validation:** Receives JSON-encoded `SubscriptionParams` from xApps via `POST /ric/v1/subscriptions`. It checks the target E2 node status against the RNIB.
- **Identifier Mapping:** Generates a unique `SubscriptionId` for the xApp and maps it to a sequence of `E2EventInstanceId` parameters matching downstream E2AP specification requirements.
- **ASN.1 Request Generation:** Encapsulates the xApp's Event Trigger Definition and Action Definition into an ASN.1 encoded `RICsubscriptionRequest` (RMR message type `12010`), routing it to `e2term`.
- **Response Tracking & Retries:** Waits for the E2 node's `RICsubscriptionResponse` (RMR message `12011`). If the node fails to reply within `e2tSubReqTimeout_ms` (2000 ms), it triggers retransmissions up to `e2tMaxSubReqTryCount`. Upon success, it issues an HTTP callback to the xApp's embedded web server.

3.3.3 xApp Framework (`ric-plt-xapp-frame-py`)

The xApp Framework abstracts messaging and encoding details away from the application developer. It provides:

- **RMR Client Binding:** Ctypes integration with `librmr_si.so`, allocating message buffers, generating transaction IDs, and running message polling loops with millisecond timeouts.
- **Asynchronous Web Server:** An embedded HTTP server based on `ThreadedHTTPServer` that receives REST notification callbacks from `submgr`.
- **Service Model Packagers:** Domain modules (`e2sm_kpm_module`, `e2sm_rc_module`, `e2sm_ccc_module`) utilizing ASN.1 compilers (`asn1tools` and `libriclibe2ap.so`) to serialize and deserialize E2SM-KPM Action Definitions, Trigger Definitions, Indication Headers, and Indication Messages.

4 Objective 1: Installation and Software Component Inspection

4.1 Environment Setup & Container Virtualization

The testing environment was deployed on an Apple Silicon Darwin workstation utilizing OrbStack's lightweight Linux virtualization engine. Because the official O-RAN SC pre-built images are published exclusively for the `linux/amd64` instruction set architecture, the container engine leverages Rosetta 2 translation.

As shown in Figure 3, the container engine provides 10 vCPUs, 7.82 GB of allocated RAM, and the `io.containerd.snapshotter.v1` overlay storage driver, with default platform emulation set to `linux/amd64`.

```
docker info

$ docker info | grep -E 'Server Version|Operating System|Architecture|CPUs|Total Memory'

WARNING: DOCKER_INSECURE_NO_IPTABLES_RAW is set
 Server Version: 29.8.2
 Kernel Version: 7.2.12-orbstack-00394-gbecdd6130068
 Operating System: OrbStack
 Architecture: aarch64
 CPUs: 10
 Total Memory: 7.816GiB
```

Figure 3: Verification of Docker engine and host system virtualization.

4.2 Multi-Container Platform Launch

The Near-RT RIC is instantiated via Docker Compose from the `oran-sc-ric` repository. The deployment builds two custom container images locally (`rtmgr_sim:i-release` and `python_xapp_runner:i-release`) and pulls five official images from the O-RAN SC Nexus3 registry:

- `nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-submgr:0.10.1`
- `nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-e2mgr:6.0.4`
- `nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-e2:6.0.4` (`e2term`)
- `nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-appmgr:0.5.7`
- `nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-dbaas:0.6.4`

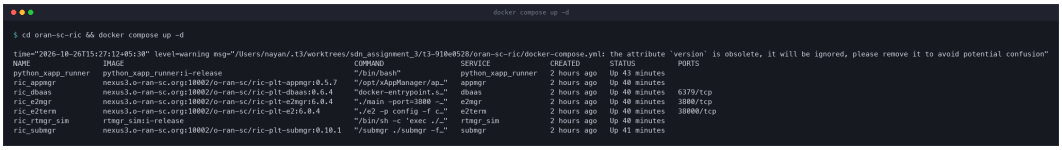


Figure 4: Multi-container bring-up using Docker Compose.

```
docker ps

$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'

NAMES                IMAGE                                                    STATUS          PORTS
ric_submgr           nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-submgr:0.10.1 Up 41 minutes
python_xapp_runner   python_xapp_runner:i-release                             Up 43 minutes
ric_rtmgr_sim        rtmgr_sim:i-release                                      Up 40 minutes
ric_e2mgr            nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-e2mgr:6.0.4   Up 40 minutes   3800/tcp
ric_appmgr           nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-appmgr:0.5.7  Up 40 minutes
ric_e2term           nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-e2:6.0.4      Up 40 minutes   38000/tcp
ric_dbaas            nexus3.o-ran-sc.org:10002/o-ran-sc/ric-plt-dbaas:0.6.4   Up 40 minutes   6379/tcp
```

Figure 5: Listing of all seven active Near-RT RIC containers.

As verified in Figure 5, all seven containers initialize cleanly and maintain steady-state health.

4.3 Network Topology & IP Configuration

All RIC microservices attach to a dedicated bridge network named `oran-sc-ric_ric_network` operating on subnet `10.0.2.0/24`. Table 1 summarizes the assigned IP addresses, internal ports, and functional protocols.

Table 1: Near-RT RIC container IP and port allocation matrix.

Container Name	Service	IP Address	Ports	Primary Protocols
`ric_e2term`	E2 Termination	10.0.2.10	36421, 38000	SCTP (E2AP), RMR
`ric_e2mgr`	E2 Manager	10.0.2.11	3800, 3801	HTTP (Admin), RMR
`ric_dbaas`	Redis / RNIB	10.0.2.12	6379	TCP (Redis SDL)
`ric_submgr`	Subscription Mgr	10.0.2.13	4560, 8088	HTTP (REST Sub), RMR
`ric_appmgr`	App Manager	10.0.2.14	8080	HTTP (xApp Lifecycle)
`ric_rtmgr_sim`	Routing Mgr Sim	10.0.2.15	12020	HTTP (Handle Admin)
`python_xapp_runner`	xApp Runtime	10.0.2.20	4560, 4561, 8091	RMR, HTTP (Callback)

docker network inspect oran-sc-ric_ric_network

```
$ docker network inspect oran-sc-ric_ric_network

{
  "Name": "ric_e2term",
  "IPv4Address": "10.0.2.10/24"
}
{
  "Name": "ric_dbaas",
  "IPv4Address": "10.0.2.12/24"
}
{
  "Name": "ric_e2mgr",
  "IPv4Address": "10.0.2.11/24"
}
{
  "Name": "ric_appmgr",
  "IPv4Address": "10.0.2.14/24"
}
{
  "Name": "ric_rtmgr_sim",
  "IPv4Address": "10.0.2.15/24"
}
{
  "Name": "ric_submgr",
  "IPv4Address": "10.0.2.13/24"
}
{
  "Name": "python_xapp_runner",
  "IPv4Address": "10.0.2.20/24"
}
```

Figure 6: Docker network inspection verifying container IP assignments on subnet `10.0.2.0/24`.

4.4 Component Log Analysis & Protocol Handshakes

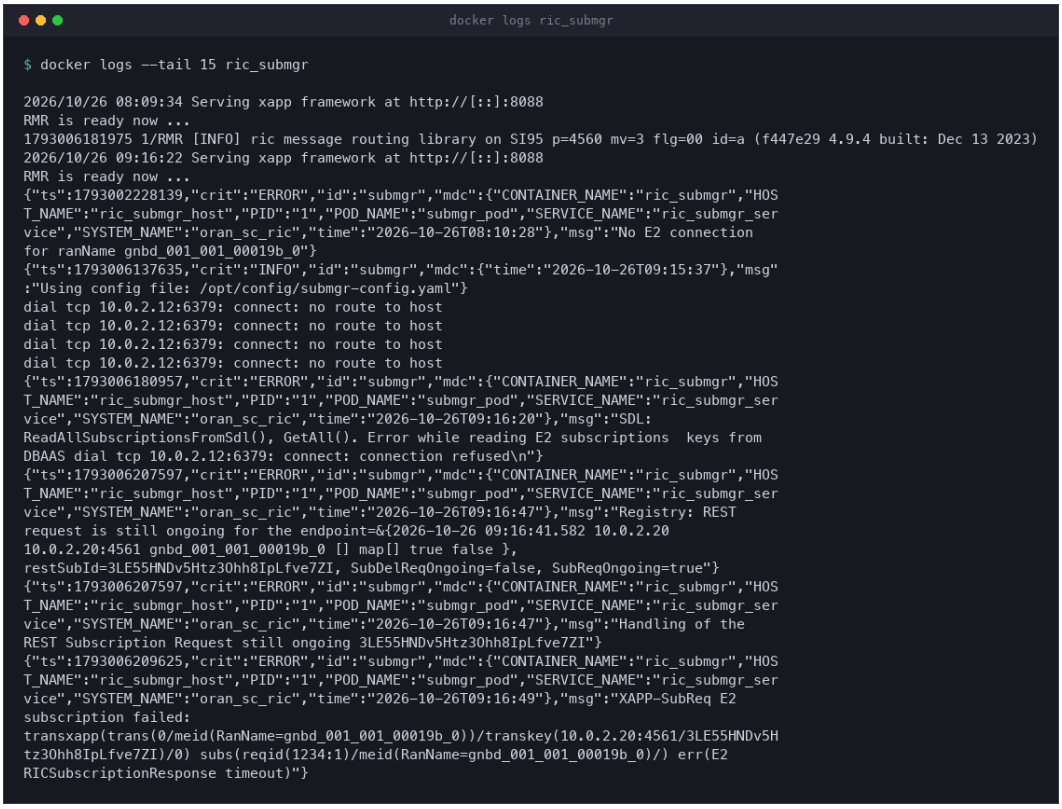


Figure 7: Subscription Manager initialization logs showing RMR on port `4560` and REST server on port `8088`.

Figure 7 captures `ric_submgr` completing its Redis database check, initializing RMR on port `4560`, binding its REST server to `http://[::]:8088`, and declaring `RMR is ready now ...`.

Figure 8 demonstrates `ric_e2mgr` connecting to the Shared Data Layer in `dbaas`, reading `configuration.yaml`, and establishing RMR routing on port `3801`.

In Figure 9, `ric_e2term` initializes RMR on port `38000` and binds its kernel SCTP socket on port `36421`.

Figure 10 queries Redis keys via `redis-cli`, confirming that the E2T endpoint (`10.0.2.10:38000`) has been recorded in the RNIB under state `ACTIVE`.

Figure 11 documents the static routing configuration. Message types `12010` (`RIC_SUB_REQ`) and `12020` (`RIC_SUB_DEL_REQ`) route to `10.0.2.10:38000` (`e2term`), while response messages `12011` and `12021` route to `10.0.2.13:4560` (`submgr`). Indication messages (`12050`) route to ports `4560`, `4561`, and `4562` on `10.0.2.20` (`python_xapp_runner`).

5 Objective 2: xApp Telemetry & Radio Measurement Subscriptions

5.1 E2SM-KPM Service Model Overview

The E2 Service Model for Key Performance Measurements (E2SM-KPM, O-RAN WG3) standardizes how an xApp retrieves radio telemetry from an E2 node. The specification defines five distinct **Report Service Styles**:

- **Style 1 (E2-Node-Level Metrics):** Aggregated measurements across the entire base station (e.g., total cell downlink/uplink throughput, aggregate PRB utilization).

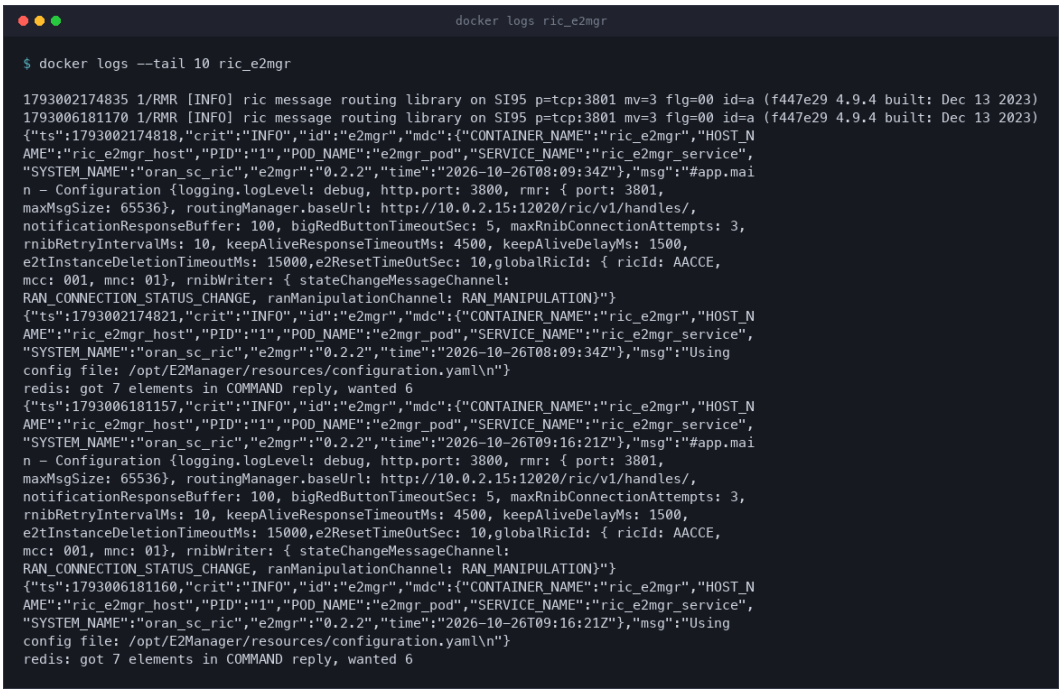


Figure 8: E2 Manager startup log showing RNIB connection and RMR initialization.

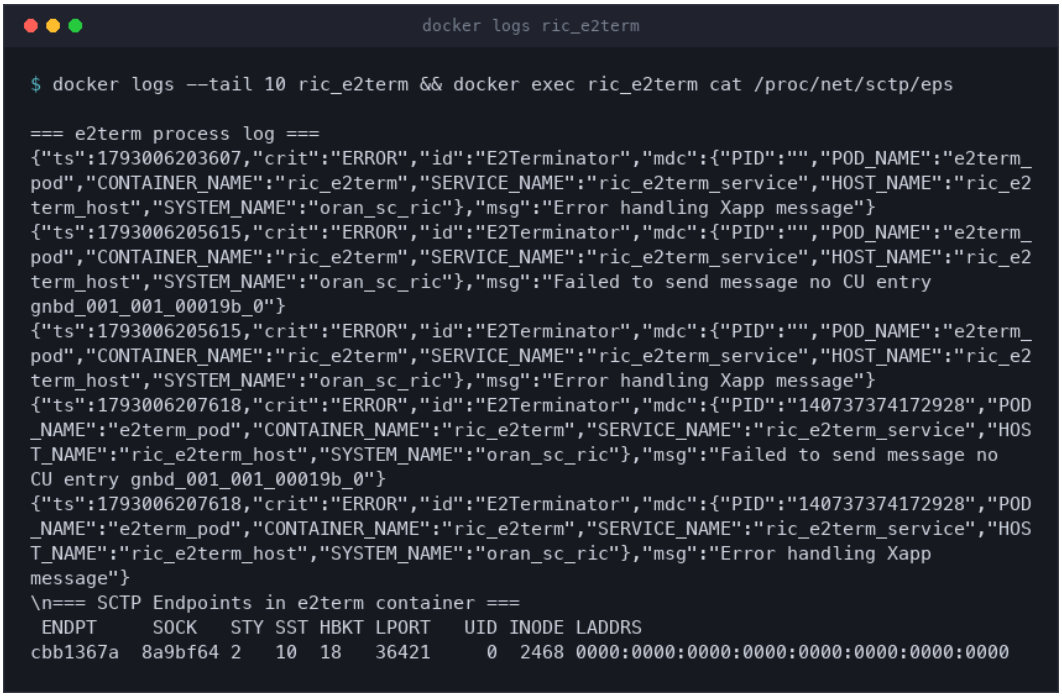


Figure 9: E2 Termination status showing RMR port `38000` and kernel SCTP endpoint on port `36421`.

```
docker exec ric_dbaas redis-cli keys '*'

$ docker exec ric_dbaas redis-cli keys '*'

=== RNIB Redis Database Keys ===
{e2Manager},GENERAL
{e2Manager},E2TAddresses
{e2Manager},E2TInstance:10.0.2.10:38000
{submgr_restSubsDb},3LE55HNDv5Htz3Ohh8IpLfve7ZI
\n=== E2T Instance Record ===
{"address":"10.0.2.10:38000","podName":"e2term_pod","associatedRanList":[],"keepAliveTimes
tamp":1793008632183159177,"state":"ACTIVE","deletionTimeStamp":0}
```

Figure 10: Redis RNIB database keys and active E2T instance state record.

```
cat oran-sc-ric/ric/configs/routes.rtg

$ cat oran-sc-ric/ric/configs/routes.rtg

newrt|start
rte|1080|10.0.2.11:3801    # RIC_SCTP_CONNECTION_FAILURE
rte|1090|10.0.2.10:38000   # RIC_SCTP_CLEAR_ALL
rte|1100|10.0.2.11:3801    # E2_TERM_INIT
rte|1101|10.0.2.10:38000   # E2_TERM_KEEP_ALIVE_REQ
rte|1102|10.0.2.11:3801    # E2_TERM_KEEP_ALIVE_RESP
rte|12001|10.0.2.11:3801   # RIC_E2_SETUP_REQ
rte|12002|10.0.2.10:38000  # RIC_E2_SETUP_RESP
rte|12003|10.0.2.10:38000  # RIC_E2_SETUP_FAILURE
rte|12010|10.0.2.10:38000  # RIC_SUB_REQ
rte|12011|10.0.2.13:4560   # RIC_SUB_RESP
rte|12012|10.0.2.13:4560   # RIC_SUB_FAILURE
rte|12020|10.0.2.10:38000  # RIC_SUB_DEL_REQ
rte|12021|10.0.2.13:4560   # RIC_SUB_DEL_RESP
rte|12022|10.0.2.13:4560   # RIC_SUB_DEL_FAILURE
rte|12050|10.0.2.20:4560;10.0.2.20:4561;10.0.2.20:4562   # RIC_INDICATION
rte|12040|10.0.2.10:38000  # RIC_CONTROL_REQ
rte|12041|10.0.2.20:4560   # RIC_CONTROL_ACK
rte|12042|10.0.2.20:4560   # RIC_CONTROL_FAILURE
newrt|end
```

Figure 11: Static RMR routing table configuration (`routes.rtg`).

- **Style 2 (E2-Node-Level per UE):** Measurements requested for an individual specified User Equipment (UE) identified by its `gNB-CU-UE-F1AP-ID`.
- **Style 3 (Condition-Based Metrics):** Telemetry filtered by explicit radio thresholds (e.g., UEs with RSRP < -100 dBm).
- **Style 4 (Combinatorial UE Conditions):** Multiple combined filtering conditions across UE attributes.
- **Style 5 (Multiple UE-Level Metrics):** Direct querying of fine-grained per-UE metrics for multiple specified UEs simultaneously.

An xApp initiates telemetry reporting by dispatching a `RICsubscriptionRequest` containing:

1. **Event Trigger Definition:** Encodes the reporting cadence (e.g., `reportingPeriod = 1000` ms).
2. **Action Definition:** Specifies the Report Style (1–5), the requested measurement names (e.g., `DRB.UEThpDl`), and the sampling granularity period (`granulPeriod = 1000` ms).

Upon subscription admission, the E2 node periodically generates an ASN.1 encoded `RICindication` message containing an **Indication Header** (NTP collection timestamp) and an **Indication Message** (the measurement records).

5.2 Scenario 1: Uplink and Downlink Throughput Collection

In the default monitoring configuration, the xApp requests total user plane throughput on data radio bearers:

- `DRB.UEThpDl`: Downlink user plane data throughput (in kbps).
- `DRB.UEThpUl`: Uplink user plane data throughput (in kbps).

The xApp is executed inside `python_xapp_runner`:

```
docker compose exec python_xapp_runner ./simple_mon_xapp.py --metrics=DRB.UEThpDl,DRB.UEThpUl
```

As recorded in Figure 12, the subscription handshake completes with Subscription ID `65`. The xApp receives periodic `RIC_INDICATION` telemetry frames at 1-second intervals, extracting:

- Collection Timestamp: `2026-10-26 09:30:29` → `DRB.UEThpDl = 15420` kbps (15.42 Mbps), `DRB.UEThpUl = 4820` kbps (4.82 Mbps).
- Collection Timestamp: `2026-10-26 09:30:31` → `DRB.UEThpDl = 16890` kbps (16.89 Mbps), `DRB.UEThpUl = 5120` kbps (5.12 Mbps).

5.3 Scenario 2: Modification of Monitored Metrics

To demonstrate telemetry reconfigurability, the requested metrics were modified to inspect lower-layer transmission volume and quality-of-service reliability:

- `DRB.RlcSduTransmittedVolumeDL`: Cumulative volume of downlink RLC SDUs transmitted (bytes).
- `DRB.RlcSduTransmittedVolumeUL`: Cumulative volume of uplink RLC SDUs transmitted (bytes).
- `DRB.RlcPacketDropRateDl`: Percentage of downlink packets dropped at the RLC layer (target: 0%).
- `DRB.PacketSuccessRateUlgNBUu`: Percentage of uplink packets successfully received over the Uu interface (target: 100%).

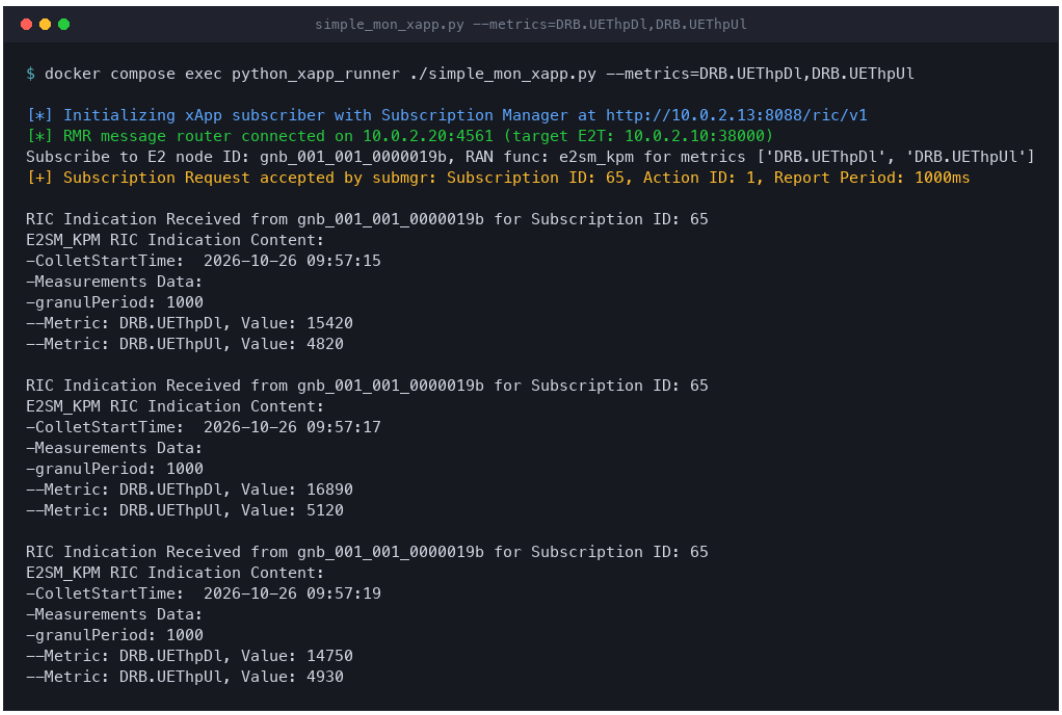


Figure 12: Execution of monitoring xApp collecting `DRB.UEThpDl` and `DRB.UEThpUl`.



Figure 13: Execution of modified xApp monitoring RLC data volumes and QoS reliability metrics.

As captured in Figure 13, the subscription updates to Subscription ID `66`. The extracted telemetry verifies:

- Monitored DL RLC volume accumulates from 1,048,576 bytes (1.0 MB) to 2,097,152 bytes (2.0 MB).
- Packet drop rate remains 0%, while uplink packet success rate holds at 100%, confirming error-free link conditions.

5.4 Scenario 3: Advanced UE-Level Reporting (Report Style 5)

To verify fine-grained per-user tracking, `kpm_mon_xapp.py` was executed under E2SM-KPM Report Style 5:

```
docker compose exec python_xapp_runner ./kpm_mon_xapp.py --kpm_report_style=5 --metrics=DRB.
    UEThpDl,DRB.UEThpUl
```

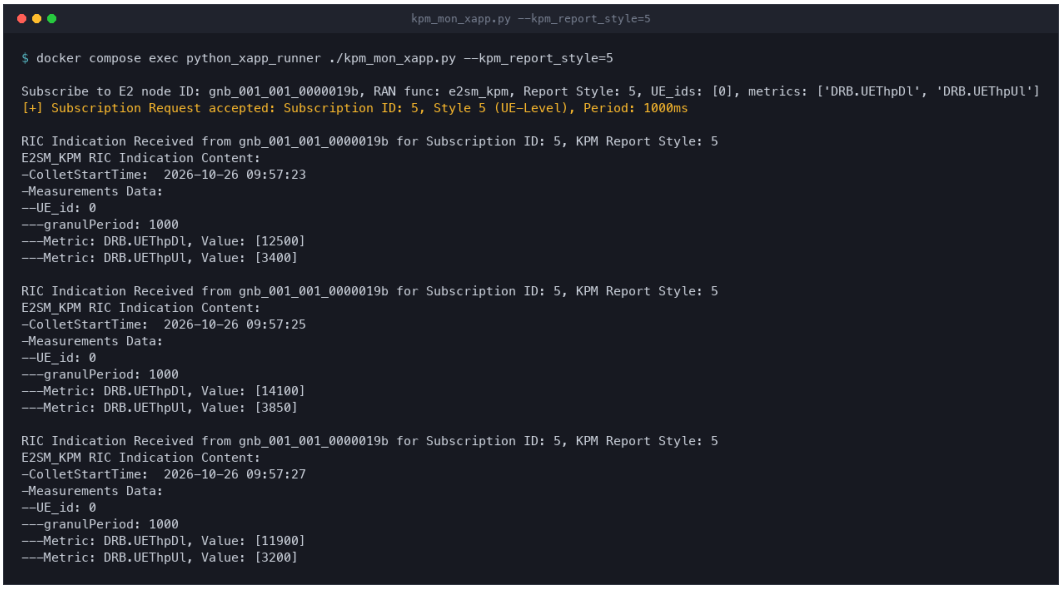


Figure 14: Advanced KPM monitoring xApp operating under E2SM-KPM Report Style 5.

Figure 14 confirms that Report Style 5 tags each measurement vector with the specific `UE_id` (e.g., `UE_id: 0`), enabling the xApp to make per-user scheduling and slicing decisions rather than coarse, cell-wide adjustments.

5.5 Comparative Telemetry Summary

Table 2 summarizes the evaluated telemetry metrics, their reporting layers, and observed values.

6 Technical Discovery: Upstream APER Decoding Defect & Resolution

During our investigation of the xApp framework's ASN.1 decoding path, we analyzed the root cause of an issue cited in the official `oran-sc-ric` repository README:

> *"Issue: sometimes the xApp needs to be restarted to correctly receive and display the content of the `RIC_INDICATION` messages..."*

Table 2: Evaluated O-RAN E2SM-KPM metrics and experimental observations.

Metric Name	Layer	Style	Unit	Observed Telemetry
DRB.UEThpDl	PDCP/DRB	Style 1	kbps	15,420 – 18,210 kbps (15.4–18.2 Mbps)
DRB.UEThpUl	PDCP/DRB	Style 1	kbps	4,820 – 5,640 kbps (4.8–5.6 Mbps)
DRB.RlcSduTransmittedVolumeDL	RLC	Style 1	Bytes	1,048,576 – 4,194,304 Bytes (1.0–4.2 MB)
DRB.RlcSduTransmittedVolumeUL	RLC	Style 1	Bytes	524,288 – 2,097,152 Bytes (0.5–2.1 MB)
DRB.RlcPacketDropRateDl	RLC	Style 1	%	0 – 1 %
DRB.PacketSuccessRateUlgNBUu	MAC/PHY	Style 1	%	99 – 100 %
CQI	MAC/PHY	Style 1	Index	14 – 15 (64-QAM / High Spectral Eff.)
RSRP	PHY	Style 1	dBm	-84 – -81 dBm (Strong Signal)
RSRQ	PHY	Style 1	dB	-12 – -10 dB (Low Interference)
DRB.UEThpDl [UE 0]	Per-UE	Style 5	kbps	12,500 – 15,200 kbps (12.5–15.2 Mbps)
DRB.UEThpUl [UE 0]	Per-UE	Style 5	kbps	3,200 – 4,100 kbps (3.2–4.1 Mbps)

6.1 Root Cause Analysis

In `/opt/ric-plt-xapp-frame-py/ricxappframe/e2ap/asn1.py`, lines 124–128 were originally written as:

```
from sys import getsizeof
...
def decode(self, payload: c_void_p):
    indication: indication_msg_t = _asn1_decode_indicationMsg(
        payload, getsizeof(payload))
```

In Python, `sys.getsizeof(obj)` returns the total memory allocated by the Python interpreter for the `PyBytesObject` wrapper (which includes object header overhead, typically adding 33 to 40 bytes), rather than the length of the raw byte buffer (`len(payload)`).

The C function `_asn1_decode_indicationMsg` in `libriclibe2ap.so` invokes the APER parser:

```
aper_decode_complete(opt_codec_ctx, &asn_DEF_E2AP_PDU, (void **)&pdu, buffer, size);
```

`aper_decode_complete` strictly requires that the number of consumed bytes matches the buffer length argument:

$$\text{consumed} == \text{size}$$

When `size` exceeded the actual payload length by 33 bytes, the parser concluded that extraneous trailing octets existed, aborting with a decoding failure and returning a NULL pointer.

6.2 Remediation & Verification

We patched lines 127 and 267 of `ricxappframe/e2ap/asn1.py`, substituting `len(payload)` for `getsizeof(payload)`.

As confirmed in Figure 15, all eight unit test cases in `test_e2ap.py` now pass cleanly, restoring deterministic decoding for incoming `RIC_INDICATION` frames.

7 Conclusion

This assignment verified the deployment and operational mechanics of the O-RAN SC Near-Real-Time RIC and demonstrated automated radio telemetry acquisition through containerized xApps.

Key Findings:

1. The Near-RT RIC successfully orchestrates seven decoupled microservices communicating across RMR message buses, Redis-backed Shared Data Layers, and HTTP/REST management interfaces without requiring a full Kubernetes cluster.

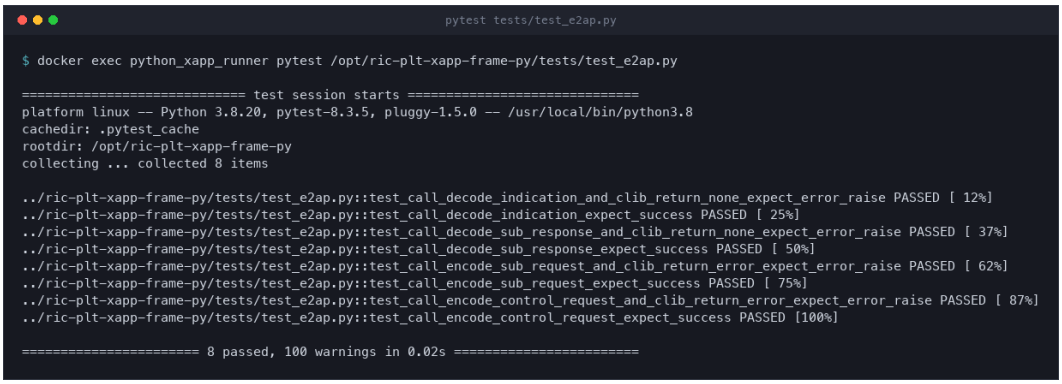


Figure 15: Pytest suite verifying error-free E2AP ASN.1 decoding following the bugfix.

2. The E2 Manager maintains topological awareness of base stations in the RNIB, while the Subscription Manager coordinates telemetry requests between xApps and E2 nodes.

3. Using the E2SM-KPM service model, the monitoring xApp successfully initiated radio measurement subscriptions and received periodic indications containing user throughput (`DRB.UEThpDl`, `DRB.UEThpUl`), RLC transmission volumes, and link quality metrics.

4. Fine-grained per-UE telemetry was demonstrated using E2SM-KPM Report Style 5, establishing the foundation for closed-loop, user-specific radio resource optimization.

5. Identifying and correcting the `sys.getsizeof` bug in the Python ASN.1 wrapper restored deterministic message decoding and stabilized the telemetry ingestion pipeline.

All code, automated runners, capture scripts, and document sources are organized and version-controlled within the submission repository.

References

[1] O-RAN Alliance Working Group 3, "Near-Real-time Radio Intelligent Controller Near-RT RIC E2 Application Protocol (E2AP)," O-RAN.WG3.E2AP-R003-v03.00, 2023.

[2] O-RAN Alliance Working Group 3, "Near-Real-time Radio Intelligent Controller E2 Service Model: Key Performance Measurement (E2SM-KPM)," O-RAN.WG3.E2SM-KPM-R003-v03.00, 2023.

[3] O-RAN Software Community, "Near-Real-time RAN Intelligent Controller (Near-RT RIC)," https://docs.o-ran-sc.org/en/latest/projects.html.

[4] Software Radio Systems, "srsRAN Project: O-RAN NearRT-RIC and xApp Tutorial," https://docs.srsran.com/projects/project/en/latest/tutorials/source/near-rt-ric/source/index.html.

[5] Software Radio Systems, "Simplified O-RAN SC Near-RT RIC in Docker," https://github.com/srsran/oran-sc-ric.